

Skyhouse Agency

API Integration:

For this step of the process I chose the Wikipedia API, to do an enrichment of the dashboard, it's free to use and you can have a short definition about the subject of the campaign and about the buzz of that topic in wikipedia website, who is constantly consulted by millions of users, so you can have a pretty precise vision about the buzz of the campaign topic.

System Design:

What tech stack would you choose and why?

A: I would go for Node.js (Nest.js) + React.js + RabbitMQ + PostgreSQL + Redis, in a monorepo with TypeScript. Nest.js gives me structure and its guards/pipes are handy to validate and authenticate webhooks early. TypeScript across the monorepo lets me share the event types between gateway, worker and React. RabbitMQ because I need a work queue with per-platform routing, dead-letter and retry control, not a full event log (I would reconsider Kafka only if we need replay or much bigger throughput). PostgreSQL for consistency and querying, with jsonb for the raw payloads. Redis for caching aggregates and pub/sub.

How would you handle webhook failures, retries, and duplicate events?

A: I would split it in two: a gateway and a worker. The gateway only verifies the platform signature (HMAC, to avoid spoofed events), validates quickly, pushes to the queue and answers 2xx fast, so the platform does not re-fire. The worker processes the event. For deduplication I would not rely on the queue; I would use an idempotency key (from the platform event id) with a unique constraint in the database. For retries I would not do blind retries: a state machine where each failure state has its policy, watching attempts and the age of the event, and deprecating old ones; what keeps failing goes to a dead-letter queue with backoff. With many workers, I claim rows with `SELECT ... FOR UPDATE SKIP LOCKED`.

How would you ensure the system scales as we add more clients and platforms?

A: Three directions. The gateway and worker are stateless, so I run more replicas and autoscale workers by queue depth. For more platforms, an adapter pattern: one adapter per platform plus a normalization layer mapping everything into a single canonical event, so a new platform is just a new adapter. For more clients, multitenant with `tenant_id` and PostgreSQL row-level security; stronger isolation (schema or database per tenant) only when really needed.

What database structure would you use to store events efficiently?

A: Two layers. A raw, append-only events table: `ids`, `tenant_id`, `platform`, idempotency key (unique index), `received` and `expire` timestamps, `status`, and a `jsonb` for the payload (hard to keep strictly typed). A second layer for the dashboard: aggregated tables or materialized views per client/platform/time bucket, because it can not scan raw `jsonb` on every load. It is time-series, so I partition by time and index on (`tenant_id`, `platform`, `received_at`). I would stay on PostgreSQL, `jsonb` already gives the flexibility, and it scales horizontally with replicas and partitioning when needed.

How would the live dashboard receive updates — polling, WebSockets, or something else?

A: Since it must be live, I would use SSE instead of polling (simpler than WebSockets, reconnects by itself), fanning out with Redis pub/sub, and pushing aggregated deltas, not every raw event. I would keep a manual refresh and an interval option only as fallback.

Bug Hunt:

Besides the bugs were already signalized in the email sent, I could add more stuff into this code.

In the calculations, I made ternary verifications to avoid divisions by zero.

In the routes I added try-catches to increase the error handling and to retrieve errors in a proper way (only the message and with the correct http status).

In the filter by minROAS route, I added a verification to check if we have the value proper as a number, if not return a bad request error, 400.

```
// =====
// SkyHouse Agency - Full Stack Developer Assessment
// Part 3: Bug Hunt
// File: campaign-dashboard.js
// This file contains FIVE intentional bugs. Find and fix them.
// =====

const express = require("express");
const fs = require("fs");
const csv = require("csv-parser");

const app = express();
app.use(express.json()); // BUG 1: express.json is a function - must be called as
express.json()
// Load campaign data from CSV
async function loadCampaignData(filePath) {
  const results = [];
  return new Promise((resolve, reject) => {
    fs.createReadStream(filePath)
      .pipe(csv())
      .on("data", (data) => results.push(data))
      .on("end", () => resolve(results))
      .on("error", (err) => reject(err));
  });
}

// Calculate ROAS for each campaign
function calculateMetrics(campaigns) {
  return campaigns.map(c => {
    const spend = parseFloat(c.spend);
    const revenue = parseFloat(c.revenue);
```

```

    const conversions = parseInt(c.conversions);
    const roas = spend > 0 ? revenue / spend : 0; // BUG 2: ROAS = revenue /
    spend, not spend / revenue
    const cpa = conversions > 0 ? spend / conversions : 0; // BUG 3: CPA = spend
    / conversions, not conversions / spend
    return { ...c, roas: roas.toFixed(2), cpa: cpa.toFixed(2) };
  });
}
// GET /api/campaigns - return all campaigns with metrics
app.get('/api/campaigns', async (req, res) => {
  try {
    const raw = await loadCampaignData("./data/campaigns.csv");
    const campaigns = calculateMetrics(raw);
    res.json({ success: true, data: campaigns });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message }); // BUG 4: Should
    send err.message, not the full Error object (exposes stack trace)
  }
});
// POST /api/campaigns/filter - filter by min ROAS
app.post('/api/campaigns/filter', async (req, res) => {
  try {
    const minRoas = parseFloat(req.body?.minRoas);
    if (Number.isNaN(minRoas)) {
      return res.status(400).json({ success: false, error: "minRoas must be a
number" });
    }
    const raw = await loadCampaignData("./data/campaigns.csv");
    const campaigns = calculateMetrics(raw);
    const filtered = campaigns.filter(c => parseFloat(c.roas) >= minRoas);
    res.json({ success: true, data: filtered });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
});
// GET /api/summary - return aggregate stats
app.get('/api/summary', async (req, res) => {
  try {
    const raw = await loadCampaignData("./data/campaigns.csv");
    const campaigns = calculateMetrics(raw);
    const totalSpend = campaigns.reduce((acc, c) => acc + parseFloat(c.spend),
0);
    const totalRevenue = campaigns.reduce((acc, c) => acc +
parseFloat(c.revenue), 0); // BUG 5: c.spend is a string from CSV - must use
parseFloat(c.spend) or it will concatenate strings

```

```
    const overallRoas = totalSpend > 0 ? (totalRevenue / totalSpend).toFixed(2)
: "0.00";
    res.json({ totalSpend, totalRevenue, overallRoas });
  } catch (err) {
    res.status(500).json({ success: false, error: err.message });
  }
});
app.listen(3000, () => console.log("Server running on port 3000"));
```